Segger v0.2.0 Release Notes

What's New in Multi-Task GNN Cell Segmentation

Segger Development Team

February 2026

What's New in v0.2.0

Core Changes:

- Multi-task loss (Triplet + Metric + Alignment)
- ME gene constraints from scRNA-seq
- Fragment mode for unassigned transcripts
- Per-gene adaptive thresholding
- Delaunay boundary construction

Engineering:

- Polars replacing Pandas/Dask
- cuSpatial replacing cuML
- Vectorized ops (10–100× speedups)
- Simplified checkpoint system
- SpatialData / AnnData / Zarr I/O
- Platform-specific quality filters

Quick Start

```
pip install segger && segger segment -i data/ -o output/
```

Segmentation as Link Prediction (Recap)

Problem: Assign each transcript t_i to its source cell c_j using spatial position and gene identity.

Heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \mathcal{T} \cup \mathcal{C}$:

tx–tx: $\mathcal{E}_{TT} = \{(t_i, t_j) : d(p_i, p_j) \leq r,\ |N(i)| \leq k\}$

tx–bd: $\mathcal{E}_{TC} = \{(t_i, c_j) : p_i \in \text{int}(B_j)\}$

bd–bd: $\mathcal{E}_{BB} = \{(c_i, c_j) : d(\bar{p}_i, \bar{p}_j) \leq r_b\}$

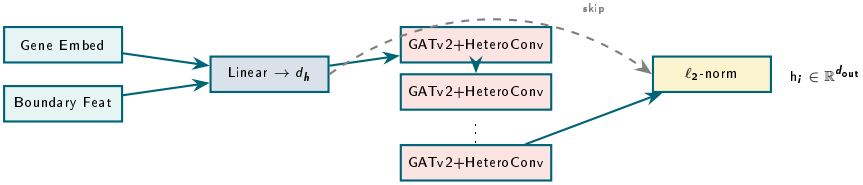
Assignment: $\hat{c}(t_i) = \arg\max_j s_{ij}$ s.t. $s_{ij} \geq \theta$
where $s_{ij} = \langle f(t_i), f(c_j) \rangle$ (L2-normalized dot product).

Boundary features $x_b \in \mathbb{R}^4$:

- Area: $A(B_i)$
- Convexity: $A(\text{CH})/A(B_i)$
- Elongation: $A(\text{MBR})/A(\text{Env})$
- Circularity: $A(B_i)/r_{\min}^2$

Gene embeddings: PCA-reduced cell-type proportion vectors from scRNA-seq.

GNN Architecture: SkipGAT (Unchanged)



GATv2 attention (per edge type ϕ):

$$\alpha_{ij}^{(\phi)} = \frac{\exp\left(\mathsf{a}^\top \mathrm{LeakyReLU}(\mathsf{W}[\mathsf{h}_i \| \mathsf{h}_j])\right)}{\sum_{k \in \mathcal{N}_\phi(i)} \exp\left(\mathsf{a}^\top \mathrm{LeakyReLU}(\mathsf{W}[\mathsf{h}_i \| \mathsf{h}_k])\right)}, \quad \mathsf{h}_i' = \sum_\phi \sum_j \alpha_{ij}^{(\phi)} \mathsf{W}_\phi \mathsf{h}_j$$

$n_{\text{mid}} + 2$ layers, each with multi-head attention. Edge types: `tx-tx`, `tx-bd`, `bd-bd`.

v0.1.0 Loss: BCE Link Prediction (Baseline)

v0.1.0 — Binary Cross-Entropy

$$\mathcal{L}_{\text{BCE}} = -\sum_{(t_i, c_j)} \left[y_{ij} \log \sigma(s_{ij}) + (1 - y_{ij}) \log(1 - \sigma(s_{ij})) \right]$$

- $s_{ij} = \langle f(t_i), f(c_j) \rangle$ (dot product of L2-normalized embeddings)
- Hard negatives from nearby cells (1:5 positive/negative ratio)
- Single global threshold θ

Limitations Addressed in v0.2.0

- Single loss — no embedding structure
- No biological priors (ME genes)
- Global threshold for all genes
- No recovery of unassigned transcripts
- Pandas/Dask data overhead
- `buffer()` only (no polygon shrink)

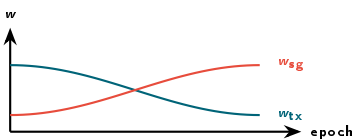
Multi-Task Loss Overview NEW

Combined Loss with Cosine-Scheduled Weights

$$\mathcal{L}_{\text{main}} = w_{\text{tx}}(t)\,\mathcal{L}^{\text{tx}}_{\text{triplet}} + w_{\text{bd}}(t)\,\mathcal{L}^{\text{bd}}_{\text{metric}} + w_{\text{sg}}(t)\,\mathcal{L}^{\text{sg}}_{\text{triplet}}$$

Weight scheduling (cosine annealing per component):

$$\alpha(t) = \tfrac{1}{2}\big(1 + \cos(\pi t/T)\big), \qquad w_i(t) = w_i^{\text{end}} + (w_i^{\text{start}} - w_i^{\text{end}})\,\alpha(t)$$



Term	Loss type	Margin
$\mathcal{L}_{\text{tx}}$	Transcript triplet	$m = 0.3$
$\mathcal{L}_{\text{bd}}$	Boundary metric	—
$\mathcal{L}_{\text{sg}}$	Segmentation triplet	$m = 0.4$

Triplet Loss + FastTripletSelector NEW

Triplet Margin Loss

$$\mathcal{L}_{\text{triplet}} = \frac{1}{|\mathcal{A}|} \sum_{(a,p,n)} \max\big(0,\ \|\mathrm{h}_a - \mathrm{h}_p\|^2 - \|\mathrm{h}_a - \mathrm{h}_n\|^2 + m\big)$$

Cluster Similarity $\mathrm{S} \in \mathbb{R}^{K \times K}$:

1. Phenograph/Louvain on boundary KNN
2. S_{ij} = Jaccard similarity of clusters
3. Positive: $P(k) \propto S_{a_c,k}$
4. Negative: $P(k) \propto -S_{a_c,k}$

FastTripletSelector — $O(1)$ per sample:

1. Build CDF: $\mathrm{cdf}_k = \sum_{j \le k} S_{a,j} / \sum S$
2. Sample $u \sim \mathrm{Unif}(0,1)$
3. $k^* = \texttt{searchsorted}(\mathrm{cdf}, u)$
4. Pick random node from cluster k^*

Complexity: $O(N)$ total vs $O(NK)$ random rejection.

Metric Loss: Boundary Embedding Quality NEW

Metric Loss (MSE on Cosine Similarity)

$$\mathcal{L}_{\text{metric}} = \mathbb{E}\big[\text{MSE}(\cos(\mathrm{h}_a, \mathrm{h}_p),\ 1 - d_{ap}) + \text{MSE}(\cos(\mathrm{h}_a, \mathrm{h}_n),\ 1 - d_{an})\big]$$

where $d_{ap} = 1 - S_{c_a, c_p}$ is cluster dissimilarity from phenograph.

Intuition:

- Similar cell embeddings $\rightarrow$ high cosine similarity
- Dissimilar cells $\rightarrow$ low cosine similarity
- Phenograph clustering = soft labels

Phenograph pipeline:

1. Cell $\times$ gene count matrix
2. PCA $\rightarrow \mathbb{R}^d$
3. KNN graph ($k = 30$)
4. Louvain ($\rho = 2$–3)
5. S: Jaccard between communities

Alignment Loss: ME Gene Constraints NEW

Contrastive Loss on tx–tx Edges

$$\mathcal{L}_{\text{align}} = \underbrace{\sum_{(i,j)\in\mathcal{P}} (1 - s_{ij})^2}_{\text{same-gene: attract}} + \underbrace{\sum_{(i,j)\in\mathcal{N}} \max(s_{ij} - m,\ 0)^2}_{\text{ME-gene: repel}}$$

$s_{ij} = \cos(\mathrm{h}_i, \mathrm{h}_j)$, $m = 0.2$, $|\mathcal{P}| \leq 3|\mathcal{N}|$ (subsample positives)

ME Gene Discovery:

1. Rank genes by expression per cell type
2. Select top 10% expressed in $\geq$30% cells
3. Filter: $\text{expr}_{\text{in}} > 0.25$, $\text{expr}_{\text{out}} < 0.03$
4. Cross-type pairs $\rightarrow$ ME set

Validation — MECR:

$$\text{MECR}(g_1, g_2) = \frac{P(g_1 \wedge g_2)}{P(g_1 \vee g_2)}$$

Lower = better. < 0.15 is good.
Alignment loss $\Rightarrow$ MECR $\downarrow$ 10–30%.

CLI Usage

```
segger segment -i data/ -o out/ -alignment-loss -scrna-reference-path ref.h5ad
```

Vectorized ME Gene Matching PERF

Problem: Given $|\mathcal{E}|$ tx–tx edges and $|\text{ME}|$ gene pairs, find ME edges efficiently.

v0.1.0: $O(|\mathcal{E}| \cdot |\text{ME}|)$

```
for i, (src, dst) in enumerate(edges):
    for g1, g2 in me_pairs:  # SLOW
        if genes[src]==g1 and genes[dst]==g2:
            is_me[i] = True
```

v0.2.0: $O(|\mathcal{E}| + |\text{ME}|)$

```
# Cantor pairing function
me_keys = me_min * G + me_max
edge_min = torch.minimum(src_g, dst_g)
edge_max = torch.maximum(src_g, dst_g)
edge_keys = edge_min * G + edge_max
is_me = torch.isin(edge_keys, me_keys)
```

Key: Map unordered pair (g_a, g_b) to scalar: key $= \min(g_a, g_b) \cdot G_{\max} + \max(g_a, g_b)$
Then `torch.isin()` for $O(1)$ amortized lookup. 10–50× speedup.

Loss Combination Modes NEW

Interpolate (Default)

$$\mathcal{L} = (1 - w_a)\mathcal{L}_{\text{main}} + w_a\mathcal{L}_{\text{align}}$$

- Total scale $\approx$ constant
- Main loss fades as alignment ramps
- Recommended for most cases

Additive

$$\mathcal{L} = \mathcal{L}_{\text{main}} + w_a\mathcal{L}_{\text{align}}$$

- Total scale increases over training
- May need LR adjustment
- For aggressive ME enforcement

Full v0.2.0 loss:

$$\mathcal{L}_{\text{v2}} = \underbrace{w_t\mathcal{L}^t_{\text{tri}}}_{\text{tx}} + \underbrace{w_b\mathcal{L}^b_{\text{met}}}_{\text{bd}} + \underbrace{w_s\mathcal{L}^s_{\text{tri}}}_{\text{seg}} + \underbrace{w_a\mathcal{L}_{\text{align}}}_{\text{ME}}$$

CLI

```
segger segment -i data/ -o out/ -loss-combination-mode interpolate
```

Fragment Mode: Recovering Unassigned Transcripts NEW

Problem: Transcripts with $\max_j s_{ij} < \theta$ remain unassigned after segmentation.

Algorithm

1. **Identify** unassigned: $\mathcal{T}_u = \{t_i : \text{seg\_cell\_id}(t_i) = \varnothing\}$
2. **Filter** tx–tx edges: $\mathcal{E}_u = \{(t_i, t_j) \in \mathcal{E}_{TT} : t_i, t_j \in \mathcal{T}_u\}$
3. **Threshold**: $\mathcal{E}'_u = \{(i,j) \in \mathcal{E}_u : s_{ij} \geq \theta_f\}$
4. **Connected components**: $\{C_1, \ldots\} = \text{CC}(\mathcal{T}_u, \mathcal{E}'_u)$
5. **Size filter**: keep C_k if $|C_k| \geq n_{\min}$ (default: 5)
6. **Assign**: $\text{seg\_cell\_id}(t_i) =$ `"frag-`k`"` for $t_i \in C_k$

Auto-threshold: Li + Yen histogram methods on similarity distribution. Fallback: median.

CLI

```
segger segment -i data/ -o out/ \
  -fragment-mode \
  -fragment-min-transcripts 10
```

GPU-Accelerated Connected Components NEW

GPU Path (RAPIDS/CuPy)

```
ids = cp.unique(cp.concat([src, dst]))
src_i = cp.searchsorted(ids, src)
dst_i = cp.searchsorted(ids, dst)

# Symmetric CSR
rows = cp.concat([src_i, dst_i])
cols = cp.concat([dst_i, src_i])
A = cp_csr((ones, (rows, cols)), (n, n))

# GPU connected components
n_cc, labels = cc_gpu(A, directed=False)
valid = cp.bincount(labels) >= n_min
```

CPU Fallback (SciPy)

```
id_map = {v: i for i, v
          in enumerate(unique_ids)}
src_i = [id_map[s] for s in src]
dst_i = [id_map[d] for d in dst]

A = csr_matrix((data, (rows, cols)),
               shape=(n, n))
n_cc, labels = cc_cpu(A, directed=False)
```

Complexity: $O(|\mathcal{T}_u| + |\mathcal{E}'_u|)$
GPU: $\sim O(|\mathcal{T}_u|/P)$ with P cores

Post-hoc similarity: Chunked cosine similarity from gene embeddings (256 MB chunks).

Per-Gene Adaptive Thresholding NEW

Fixed Threshold (v0.1.0)

$$\hat{c}(t_i) = \begin{cases} \arg\max_j s_{ij} & \max_j s_{ij} \geq \theta \\ \varnothing & \text{otherwise} \end{cases}$$

Single θ for all genes.

Per-Gene Auto (v0.2.0)

For each gene g:

$$\theta_g = \min(\theta_g^{\text{Li}}, \theta_g^{\text{Yen}})$$

Fallback: $\theta_g = \text{median}(s_g)$.

Li: Minimize cross-entropy

$$\theta^{\text{Li}} = \arg\min_\theta \sum_x p(x) \log \frac{p(x)}{q_\theta(x)}$$

Yen: Maximum correlation

$$\theta^{\text{Yen}} = \arg\max_\theta \frac{(\sum_{x \leq \theta} x p(x))^2}{\sum_{x \leq \theta} p(x)}$$

Memory: $|S_g| > 10^7 \Rightarrow$ subsample.
Both methods robust to multi-modal distributions.

CLI — Fixed vs Auto

```
segger segment -i data/ -o out/ -min-similarity 0.5  (fixed)
segger segment -i data/ -o out/  (auto per-gene, default)
```

Two-Phase Delaunay Boundary Extraction NEW

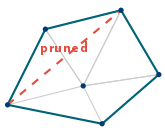
Construct cell boundaries de novo from transcript point clouds.

Phase 1: Long Edge Removal

1. $d_{\max}$ = max nearest-neighbor distance
2. Find boundary edges (< 2 incident simplices)
3. Remove if $\ell > 2\, d_{\max}$
4. Propagate: deleted simplices $\rightarrow$ new boundary

Phase 2: Extreme Angle Removal

$$\text{Remove if } \begin{cases} \ell > 1.5 d_{\max} \wedge \alpha > 90^\circ \\ \text{or } \alpha > 168.75^\circ \end{cases}$$



Cycle detection: DFS on boundary graph $\rightarrow$ Shapely polygons.
Complexity: $O(n \log n)$ (Delaunay dominates).
Angles: Vectorized — all triangles at once.

Polars Replacing Pandas/Dask CHANGED

Why Polars:

- Lazy evaluation + query optimization
- Streaming mode for >10M rows
- 2–3× less memory than Pandas
- Apache Arrow columnar backend
- Rust-based parallelism

```
# Auto-streaming for large files
lf = pl.scan_parquet("transcripts.pq")
result = lf.filter(
    pl.col("qv") >= 20
).group_by("cell_id").agg(
    pl.count()
).collect(streaming=True)
```

cuSpatial Replacing cuML:

- GPU-native point-in-polygon
- Quadtree spatial indexing
- Polygon area/centroid ops

GPU/CPU Fallback

cuSpatial → GeoPandas
CuPy sparse → SciPy sparse
cuML → scanpy/sklearn
All GPU ops auto-fallback to CPU.

Vectorized Batch Normalization PERF

Positional encoding: Sinusoidal 2D spatial embeddings:

$$\mathrm{PE}(p, 2k) = \sin\left(p/10000^{2k/d}\right), \quad \mathrm{PE}(p, 2k{+}1) = \cos\left(p/10000^{2k/d}\right)$$

v0.1.0 — per-sample loop, $O(B \cdot N \cdot d)$:

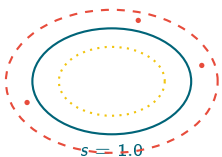
```
for b in range(num_batches):  # SLOW
    mask = (batch == b)
    pos[mask] = (pos[mask] - pos[mask].min(0)
                 ) / (pos[mask].max(0)
                      - pos[mask].min(0))
```

v0.2.0 — scatter ops, $O(N \cdot d)$:

```
mins, _ = scatter_min(pos, batch,
                      dim=0, dim_size=B)
maxs, _ = scatter_max(pos, batch,
                      dim=0, dim_size=B)
pos_norm = (pos - mins[batch]) / (
    maxs[batch] - mins[batch]
).clamp_min(1e-8)
```

10–100× speedup. Eliminates per-batch Python loop entirely.

Polygon Scaling: `scale()` vs `buffer()` CHANGED



v0.1.0: `buffer()` — isotropic dilation only
v0.2.0: `scale()` — expand **and** shrink

$$B_j^{(s)} = \text{scale}(B_j, s), \quad s \in (0, \infty)$$

Recommended settings:

- Training: $s = 1.0$
- Prediction: $s = 1.0$–1.5
- High-precision: $s < 1.0$

CLI

```
segger segment -i data/ -o out/ -prediction-scale-factor 1.2
```

Simplified Checkpoint Workflows NEW

Mode	Description	Vocabulary Handling
`train`	Fresh training	Embeddings from data vocab
`finetune`	Load weights, reset optim	Auto-remap: shared → keep, new → Xavier
`predict`	Skip training	Auto-filter data to checkpoint genes

Vocab remapping (finetune):

$$V_{\text{shared}} = V_{\text{ckpt}} \cap V_{\text{data}}$$
$$V_{\text{new}} = V_{\text{data}} \setminus V_{\text{ckpt}}$$

Gene embeddings exported to `gene_embeddings.parquet` by default.

CLI Examples

```
# Predict from checkpoint
segger segment -i data/ -o out/ \
  --checkpoint-path model.ckpt \
  --checkpoint-mode predict

# Finetune on new dataset
segger segment -i data/ -o out/ \
  --checkpoint-path model.ckpt \
  --checkpoint-mode finetune
```

Removed: `-vocab-mode`, `-checkpoint-vocab`, `-filter-to-checkpoint-vocab` (all auto).

I/O: SpatialData, AnnData, Quality Filters NEW

Output formats:

- `segger_raw`: Parquet predictions
- `merged`: Transcripts + assignments
- `spatialdata`: SpatialData Zarr (SOPA-compatible)
- `anndata`: Cell × gene `.h5ad`
- `all`: Everything

Platform quality filters:

Platform	Field	Default
Xenium	qv	≥ 20
CosMx	Controls	Remove
MERSCOPE	Blanks	Remove

CLI Examples

```
# Output as SpatialData Zarr
segger segment -i data/ -o out/ \
  --output-format spatialdata

# Export to Xenium Explorer
segger export \
  -s predictions.parquet \
  -i /path/to/xenium/ \
  -o /export/ --num-workers 4

# Input from SpatialData
segger segment \
  -i data.zarr \
  --input-format spatialdata \
  -o out/
```

Vectorization & Scalability Summary

Operation	v0.1.0	v0.2.0	Speedup
Batch norm	Per-sample loop	`scatter_min/max`	10–100×
ME matching	Per-edge loop	Cantor hash + `isin`	10–50×
Triplet sampling	Random rejection	FastTripletSelector	∼5×
Data loading	Pandas + Dask	Polars LazyFrames	2–3×

Training features:

- Mixed precision: `16-mixed`, `bf16-mixed`
- LR schedulers: cosine, OneCycle
- Early stopping
- Tile-based: 50K–100K tx/tile

GPU/CPU dual path:

- CuPy → SciPy (fragment CC)
- cuSpatial → GeoPandas (geometry)
- cuML → scanpy (clustering)
- Auto 3D detection

CLI — Training Options

```
segger segment -i data/ -o out/ -precision 16-mixed -lr-scheduler cosine
```

Optuna-Based Hyperparameter Optimization NEW

Features:

- Multi-objective: up to 5 objectives
- Samplers: TPE, NSGA-III, CMA-ES, Random
- Pruner: Hyperband (3× faster)
- SQLite persistence (resume-able)
- Multi-fidelity (10–20% data first)

Search space:

- Architecture: channels, layers, heads
- Training: LR, scheduler, loss
- Graph: k, dist, scale factor
- Loss: weights, margins

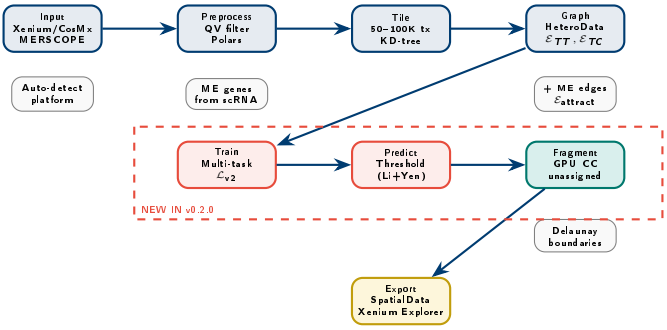
CLI Examples

```
# Basic HPO (50 trials)
segger hpo -i data/ -o results/ \
  --n-trials 50

# Multi-objective with NSGA-III
segger hpo -i data/ -o results/ \
  --n-objectives 5 --sampler nsga3

# Persistent study (resume-able)
segger hpo -i data/ -o results/ \
  --storage sqlite:///hpo.db

# Smart two-stage workflow
segger hpo -i data/ -o results/ \
  --workflow smart --n-trials 50
  # Stage 1: 50 trials on 20% data
  # Stage 2: top 10 on full data

# View results
optuna-dashboard sqlite:///hpo.db
```

v0.1.0 vs v0.2.0: Complete Comparison

Feature	v0.1.0	v0.2.0
Data processing	Pandas + Dask	**Polars** (2–3× faster)
GPU geometry	RAPIDS cuML	**cuSpatial** (native PiP)
Loss function	BCE only	**BCE + Triplet + Metric + Alignment**
Triplet sampling	Random	**FastTripletSelector** (inverse CDF)
Biological priors	None	**ME gene alignment loss**
Gene embeddings	One-hot / external	**Learned + PCA + exported**
Batch normalization	Per-sample loop	**scatter_min/max** (10–100×)
Polygon scaling	`buffer()` only	`scale()` (expand + shrink)
Similarity threshold	Global θ	**Per-gene Li + Yen auto**
Unassigned transcripts	Discarded	**Fragment mode** (GPU CC)
Boundary construction	External only	**Delaunay** (de novo)
Checkpoint system	Manual vocab	**Auto** (train/finetune/predict)
Weight scheduling	Fixed	**Cosine annealing**
Export formats	Parquet	**Zarr, SpatialData, AnnData, Explorer**
Quality filtering	Manual	**Platform-specific** (Xenium/CosMx/MERSCOPE)
HPO	None	**Optuna** (multi-objective, multi-fidelity)
Training precision	FP32 only	**FP16/BF16 mixed precision**

v0.2.0 Pipeline Overview



Key Equations Summary

Assignment: $$\hat{c}(t_i) = \arg\max_j \langle f(t_i), f(c_j) \rangle \text{ s.t. } s_{ij} \geq \theta_g \tag{1}$$

BCE (v0.1): $$\mathcal{L}_{\text{BCE}} = -\sum \left[y \log \sigma(s) + (1-y) \log(1-\sigma(s)) \right] \tag{2}$$

Triplet: $$\mathcal{L}_{\text{tri}} = \frac{1}{|\mathcal{A}|} \sum \max\left(0, \|h_a - h_p\|^2 - \|h_a - h_n\|^2 + m\right) \tag{3}$$

Metric: $$\mathcal{L}_{\text{met}} = \mathbb{E}\left[\text{MSE}(\cos(h_a, h_p), 1-d) + \text{MSE}(\cos(h_a, h_n), 1-d) \right] \tag{4}$$

Alignment: $$\mathcal{L}_{\text{align}} = \sum_{\mathcal{P}} (1-s)^2 + \sum_{\mathcal{N}} \max(s-m, 0)^2 \tag{5}$$

v0.2.0: $$\mathcal{L} = (1-w_a)\left[w_t \mathcal{L}^t_{\text{tri}} + w_b \mathcal{L}^b_{\text{met}} + w_s \mathcal{L}^s_{\text{tri}} \right] + w_a \mathcal{L}_{\text{align}} \tag{6}$$

Schedule: $$w_i(t) = w_i^{\text{end}} + (w_i^{\text{start}} - w_i^{\text{end}}) \cdot \tfrac{1}{2}(1 + \cos(\pi t / T)) \tag{7}$$

Segger v0.2.0

Install & Get Started

Install:

```
pip install segger
```

Basic segmentation:

```
segger segment -i data/ -o out/
```

With ME gene alignment:

```
segger segment -i data/ -o out/ \
        -alignment-loss \
    -scrna-reference-path ref.h5ad
```

Export to Xenium Explorer:

```
segger export -s seg.pq -o exp/
```

From checkpoint:

```
segger segment -i data/ -o out/ \
    -checkpoint-path model.ckpt \
    -checkpoint-mode predict
```

HPO:

```
segger hpo -i data/ -o hpo/
```

Segger: scalable graph neural network cell segmentation (2025)

Backup: Hyperparameter Recommendations

Parameter	Heuristic	Range
`transcripts_max_k`	90th pctl of local density	5–15
`transcripts_max_dist`	$\approx 0.5\times$ median nuclear radius	5–10 μm
`prediction_scale_factor`	Expand for edge tx	1.0–1.5
`tile_size`	50K–100K transcripts	By VRAM
`hidden_channels`	Model capacity	32–128
`n_heads`	Attention diversity	3–8
`alignment_loss_weight_end`	ME strength	0.05–0.2
`fragment_min_transcripts`	Min component size	5–20
`learning_rate`	With scheduler	10^{-4}–10^{-3}

Backup: FastTripletSelector Algorithm

FastTripletSelector

Input: Embeddings H, cluster labels c, similarity $S \in \mathbb{R}^{K \times K}$

Output: Triplets (a, p, n) for each anchor

```
# 1. Build per-cluster index
counts = bincount(c); offsets = cumsum(counts); sorted_idx = argsort(c)

# 2. Build CDFs for positive/negative sampling
pdf_pos[a, k] = S[c_a, k] / sum(S[c_a, :])      # for present clusters
cdf_pos = cumsum(pdf_pos, axis=1)
pdf_neg[a, k] = max(-S[c_a, k], 0) / sum(max(-S[c_a, :], 0))
cdf_neg = cumsum(pdf_neg, axis=1)

# 3. Sample triplets (vectorized over all anchors)
for each anchor a:
    u_p ~ Unif(0,1);  k_p = searchsorted(cdf_pos[a], u_p)
    p = sorted_idx[offsets[k_p] + floor(rand() * counts[k_p])]
    u_n ~ Unif(0,1);  k_n = searchsorted(cdf_neg[a], u_n)
    n = sorted_idx[offsets[k_n] + floor(rand() * counts[k_n])]
return triplets (a, p, n)
```

Complexity: $O(N)$ total via inverse CDF sampling ($O(1)$ per anchor).

Backup: Quality Filtering by Platform

Platform	Quality Field	Filter	Threshold
Xenium (10X)	`qv` (Phred)	$\text{qv} \geq \theta$	20 ($\equiv$ 1% error)
CosMx (NanoString)	Control probes	Remove controls	Boolean
MERSCOPE (Vizgen)	Blank codes	Remove `BLANK_*`	Prefix

Phred quality: $Q = -10 \log_{10}(P_{\text{error}}) \quad \Rightarrow \quad$ At $Q = 20$: $P_{\text{error}} = 0.01$ (99% confidence).

Auto-detection: Platform inferred from column names (`qv`, `cell_id`, `fov`, `codeword_index`).

Backup: Complete CLI Reference

Segmentation:

```
segger segment -i data/ -o out/ \
  --min-similarity 0.5 \
  --prediction-scale-factor 1.2 \
  --precision 16-mixed \
  --lr-scheduler cosine \
  --alignment-loss \
  --scrna-reference-path ref.h5ad \
  --loss-combination-mode interpolate \
  --fragment-mode \
  --fragment-min-transcripts 10 \
  --output-format spatialdata \
  --use-3d auto --min-qv 20
```

From checkpoint:

```
segger segment -i data/ -o out/ \
  --checkpoint-path model.ckpt \
  --checkpoint-mode predict
```

Export:

```
segger export \
  -s predictions.parquet \
  -i /path/to/xenium/ \
  -o /export/ --num-workers 4
```

HPO:

```
segger hpo -i data/ -o hpo/ \
  --n-trials 50 --sampler nsga3 \
  --n-objectives 5 \
  --storage sqlite:///hpo.db \
  --workflow smart
```

Tests:

```
pip install segger[hpo]
pytest tests/ -v
```